

SQL Concepts Mini-Report

content	pages
SQL Transactions	2 - 3
SQL Stored Procedures	4 - 5
Summary	6

Note:

This report was prepared after understanding the concepts of **Transactions** and **Stored Procedures** in databases. The main objective of this report is to summarize these concepts **in my own concise way** and to **apply them practically using SQL based on my understanding**, to connect the theoretical concepts with real-world database operations.

In SQL, **transactions** and **stored procedures** are essential for maintaining data integrity and automating repetitive tasks.

- **Transactions** ensure that a series of SQL operations either **all succeed or all fail**, preventing partial updates that could corrupt data.
- **Stored Procedures** are **predefined SQL queries or scripts** stored in the database. They can accept inputs, execute multiple operations, and return results.

SQL Transactions

1. What is Transaction?

A **transaction** is a logical unit of work that contains one or more SQL statements. A transaction follows the **ACID properties**:

1. **Atomicity**: All operations in a transaction succeed or fail together.
2. **Consistency**: The database moves from one valid state to another.
3. **Isolation**: Transactions are independent and do not interfere with each other.
4. **Durability**: Once committed, changes are permanent.

2. Transaction Commands

- **BEGIN TRANSACTION** – Starts a transaction.
- **COMMIT** – Saves all changes made in the transaction.
- **ROLLBACK** – Reverts all changes if there is an error.

3. Transaction Example

```
SQLQuery1.sql - A...ANOUD\anoud (61)) *  X
-- Insert data
INSERT INTO Accounts VALUES (1, 'Anoud', 1000.00)
INSERT INTO Accounts VALUES (2, 'Ahmed', 500.00)

BEGIN TRANSACTION

UPDATE Accounts      -- deduction from fist account
SET Balance = Balance - 20
WHERE AccountID = 1;

UPDATE Accounts      -- add to account 2
SET Balance = Balance + 20
WHERE AccountID = 2;

COMMIT      -- this transaction successfull

select * from Accounts
-----
BEGIN TRANSACTION

UPDATE Accounts      --deduction
SET Balance = Balance - 200
WHERE AccountID = 1

UPDATE Accounts      -- wrong account number
SET Balance = Balance + 200
WHERE AccountID = 9 -- this account not available

ROLLBACK
PRINT 'Wrong Account Number - Transaction Rolled Back'
END
```

107 %

Results Messages

	AccountID	AccountHolder	Balance
1	1	Anoud	-720.00
2	2	Ahmed	1120.00

SQL Stored Procedures

1. What is a Stored Procedure?

A **stored procedure** is a **saved collection of SQL statements** that can:

- Accept parameters
- Perform multiple operations
- Return results

Benefits:

- Reusability
- Security (users can execute procedures without direct table access)
- Performance (precompiled in the database)

2. Stored Procedures Example

```
SQLQuery1.sql - A...ANOUD\anoud (61))* -p X
CREATE PROCEDURE sp_TransferMoney
    @FromAccountID INT,
    @ToAccountID INT,
    @Amount DECIMAL(10,2)
AS
BEGIN
    SET NOCOUNT ON

    BEGIN TRANSACTION
    BEGIN TRY
        UPDATE Accounts
        SET Balance = Balance - @Amount
        WHERE AccountID = @FromAccountID;

        IF @@ROWCOUNT = 0
            THROW 50001, 'From Account not found', 1

        UPDATE Accounts
        SET Balance = Balance + @Amount
        WHERE AccountID = @ToAccountID;

        IF @@ROWCOUNT = 0
            THROW 50002, 'To Account not found', 1

        COMMIT

        SELECT 'Transaction Successful' AS Message;
    END TRY
    BEGIN CATCH
        ROLLBACK

        SELECT
            'Transaction Failed' AS Message,
            ERROR_MESSAGE() AS ErrorDetails;
    END CATCH
END
```

Validation:

```
-----Successful Transfer
EXEC sp_TransferMoney
    @FromAccountID = 1,
    @ToAccountID = 2,
    @Amount = 50
```

107 %

Results Messages

	Message
1	Transaction Successful

```
-----Failed Transfer (Wrong Account)
EXEC sp_TransferMoney
    @FromAccountID = 1,
    @ToAccountID = 9,
    @Amount = 50
```

107 %

Results Messages

	Message	ErrorDetails
1	Transaction Failed	To Account not found

Summary: SQL Concepts in Banking Use Case

	explanation	example
Stored Procedures	Predefined SQL scripts to perform tasks	Add or query customer accounts. EXEC GetCustomerAccounts @CustomerID=101
Transactions	Ensure multi-step operations are safe and consistent	Money transfer between accounts: - Success → COMMIT - Error → ROLLBACK
NTILE Function	Divides data into roughly equal groups (tiles) and assigns a group number	Divide customers into 4 groups by account balance: - Top 25% → Quartile 1 - Bottom 25% → Quartile 4

